KNOWLEDGE TRANSFER DOCUMENT

Project: E-Commerce Order Management System

Project Type: Real-Time Enterprise Web Application
Domain: E-Commerce
Application: Online Shopping Platform
Technology: Java, Spring Boot, React, MySQL, REST APIs
Version Control: Git
Deployment: AWS / Cloud Environment
Environments: Development, QA, UAT, Production

---

1. Project Introduction

The E-Commerce Order Management System is a web-based application used by customers to browse products, add products to the shopping cart, place orders, make payments, and track deliveries.

The main purpose of the project is to automate the complete order lifecycle from the time a customer selects a product until the order is delivered.

The system also provides backend services for managing products, customers, orders, payments, inventory, and delivery information.

---

2. Business Requirement

The business wants to provide customers with an online shopping platform where they can purchase products without visiting a physical store.

Customers should be able to register, log in, search for products, view product details, add products to the cart, place orders, make payments, and track their orders.

The business team also needs an administrative system through which employees can manage products, inventory, orders, and customers.

---

3. Real-Time Business Scenario

Consider a customer who wants to purchase a mobile phone.

The customer logs into the website and searches for the mobile phone. After checking the price and availability, the customer adds it to the cart.

The customer then provides the delivery address and proceeds to payment. After successful payment, an order is created and the inventory quantity is reduced.

The customer receives an order confirmation and can later track the order until it is delivered.

This complete process is handled by different modules of the application.

---

4. Project Architecture

The application follows a layered architecture.

The frontend is responsible for displaying the user interface. It communicates with the backend through REST APIs.

The Spring Boot backend contains the business logic. It communicates with the MySQL database and external services such as payment and notification services.

The general architecture is:

```
Customer
   |
   v
React Frontend
   |
   v
REST APIs
   |
   v
Spring Boot Backend
   |
   +-------------------+
   |         |         |
   v         v         v
Product   Order      Payment
Service   Service    Service
   |         |         |
   +---------+---------+
             |
             v
          MySQL
```

---

5. Technology Stack

The frontend is developed using React.

Java with Spring Boot is used for backend development because it provides a structured framework for creating enterprise applications and REST APIs.

MySQL is used as the relational database.

Git is used for source-code management.

The application can be deployed to a cloud environment such as AWS.

---

6. Project Environments

The application is maintained in multiple environments.

The development environment is used by developers while implementing features.

The QA environment is used by testers to validate functionality.

The UAT environment is used for business validation.

Production is the live environment accessed by customers.

The normal deployment flow is:

```
Development → QA → UAT → Production
```

Each environment may have different configuration values and databases.

---

7. User Registration and Login

Customers first register with the application by providing the required information.

After registration, the customer can log in using valid credentials.

The backend validates the credentials and creates an authenticated session or token.

Authentication is important because customers should only be able to access their own account and order information.

8. Customer Profile Module

The customer profile module stores information such as name, email, mobile number, and delivery addresses.

Customers can update permitted profile information through the application.

The backend validates the request before updating the database.

Customer information must be handled securely because it contains personal data.

---

9. Product Module

The product module manages products available for purchase.

A product can contain information such as product name, description, category, price, image, and availability.

The product module provides APIs that allow the frontend to retrieve product information.

Administrators can add, update, or deactivate products according to their permissions.

---

10. Product Search

Customers need to find products quickly.

The search functionality allows users to search using product names or keywords.

The backend receives the search request and queries the database.

For a large application, search performance becomes important, so indexes or dedicated search technologies may be introduced.

---

11. Product Details

When a customer selects a product, the application displays detailed information.

This may include product name, description, price, available quantity, specifications, and customer reviews.

The frontend obtains this information through a backend API.

The backend verifies that the product is active before returning the details.

---

12. Shopping Cart

The shopping cart stores products selected by the customer before placing an order.

The customer can increase or decrease quantities or remove products from the cart.

The backend calculates the current cart value using product and pricing information.

The system should validate product availability before allowing the customer to proceed to checkout.

---

13. Inventory Management

Inventory management tracks the quantity of products available for sale.

For example, if a product has 20 units available and a customer purchases 2 units, the available quantity should be updated according to the application's inventory rules.

Inventory management is important because the system should prevent customers from purchasing products that are no longer available.

---

14. Checkout Process

Checkout is the stage where the customer confirms the order.

The customer selects the delivery address, reviews the products, checks the total amount, and selects a payment method.

Before creating the final order, the backend validates the cart, pricing, inventory, and customer information.

---

15. Order Creation

After successful checkout validation, the application creates an order.

An order normally receives a unique order number.

For example:

```
ORD-2026-100245
```

The order contains information such as customer, products, quantity, amount, delivery address, payment status, and order status.

---

16. Order Lifecycle

An order can move through different statuses.

For example:

```
PLACED
   ↓
CONFIRMED
   ↓
PACKED
   ↓
SHIPPED
   ↓
OUT_FOR_DELIVERY
   ↓
DELIVERED
```

If an order is cancelled, the status may become `CANCELLED`.

The exact lifecycle depends on the business requirements.

---

17. Payment Processing

Payment processing is one of the important parts of the application.

The customer selects a payment method and submits the payment request.

The application communicates with a payment service or gateway.

The payment response determines whether the order can proceed.

The application must carefully handle successful, failed, pending, and timed-out payment situations.

---

18. Payment Failure Scenario

Suppose a customer attempts to purchase a product for ₹5,000.

The payment gateway does not respond because of a temporary network problem.

The application should not simply assume that the payment failed.

The system may need to check the payment status or follow a reconciliation process before deciding the final order status.

This type of situation is common in real-world payment systems.

---

19. Order Database

The order information is stored in the database.

A simplified order table may contain:

```
ORDER
-------------------------
order_id
customer_id
order_date
total_amount
payment_status
order_status
delivery_address
created_at
updated_at
```

The order items are normally stored separately because one order can contain multiple products.

---

20. Order Item Table

The order item table can contain:

```
ORDER_ITEM
-------------------------
order_item_id
order_id
```

```
product_id
quantity
price
subtotal
```

The relationship is:

```
ORDER
  |
  | 1
  |
  | N
ORDER_ITEM
```

This allows a single order to contain multiple products.

---

21. REST APIs

The frontend communicates with the backend through REST APIs.

Some example APIs are:

```
POST   /api/login
GET    /api/products
GET    /api/products/{id}
POST   /api/cart
GET    /api/cart
POST   /api/orders
GET    /api/orders/{id}
POST   /api/payments
```

Each API performs a specific business operation.

---

22. Backend Structure

The Spring Boot application follows a layered structure.

A typical project structure is:

```
src/main/java
 |
 +-- controller
 |
 +-- service
 |
 +-- repository
 |
 +-- model
 |
 +-- exception
 |
 +-- configuration
```

The controller receives requests, the service contains business logic, and the repository communicates with the database.

---

23. Controller Layer

The controller is responsible for handling HTTP requests.

For example, when a customer places an order, the frontend sends a request to the order controller.

The controller validates the request format and passes it to the service layer.

Business logic should generally not be placed directly inside the controller.

---

24. Service Layer

The service layer contains the main business logic.

For order creation, the service may validate:

- Customer information
- Cart contents
- Product availability
- Product price
- Delivery address
- Payment information

After successful validation, it creates the order and related records.

---

25. Repository Layer

The repository layer communicates with MySQL.

It is responsible for retrieving and storing information such as:

- Customers
- Products
- Orders
- Order items
- Payments

Spring Data JPA can be used to simplify database access.

---

26. Exception Handling

Applications can experience many errors.

For example, a product may no longer exist, the database may be unavailable, or an external payment service may return an error.

The application should handle these exceptions properly and return meaningful responses to the frontend.

Technical error details should not be exposed unnecessarily to customers.

---

27. Logging

Logging is important for troubleshooting.

The application can log events such as:

```
Order creation started
Payment request sent
Payment response received
Order status updated
Database error occurred
```

Logs help developers understand what happened when an issue occurs in production.

Sensitive information such as passwords and payment credentials should not be written to logs.

---

28. Security

Security is important because the application handles customer information and payment-related operations.

The system should use HTTPS, secure authentication, authorization, input validation, password hashing, and proper access control.

API endpoints should verify that the logged-in user has permission to access the requested information.

---

29. Git Workflow

Developers use Git to manage source code.

A typical workflow is:

```
Create Branch
     ↓
Write Code
     ↓
Run Tests
     ↓
Commit
     ↓
Push
     ↓
Pull Request
     ↓
Code Review
     ↓
Merge
```

For example, a developer working on order cancellation might create:

```
feature/order-cancellation
```

---

30. Code Review

Before code is merged, another developer reviews it.

The reviewer checks whether the implementation follows project standards and whether the business logic is correct.

Security, error handling, performance, readability, and test coverage are also considered.

Code review helps reduce defects before changes reach QA.

---

31. Testing

The application is tested at different levels.

Developers perform unit testing for individual components.

QA performs functional and integration testing.

API testing is used to verify backend endpoints.

Regression testing ensures that new changes have not broken existing functionality.

Performance and security testing may also be performed for important releases.

---

32. Real-Time Bug Scenario

Suppose QA reports:

"Order total is different between the cart and checkout page."

The developer first reproduces the issue.

Then the developer checks the frontend calculation, API response, database values, discount logic, and pricing rules.

After identifying the cause, the developer fixes the code and adds an appropriate test.

The change is committed and sent for code review before moving back to QA.

---

33. CI/CD Pipeline

After code is merged, the CI/CD pipeline automatically builds the application.

A simplified pipeline is:

```
Git Push
   ↓
Build
   ↓
Unit Tests
   ↓
Code Quality Check
   ↓
Package Application
   ↓
Deploy to QA
```

If the pipeline fails, developers investigate the failure before proceeding.

---

34. Production Deployment

After QA and business validation, the application can be released to production according to the organization's approval process.

Production deployment should be carefully planned because real customers use the application.

After deployment, the team monitors application health, errors, response times, and important business transactions.

---

35. Production Support

After deployment, developers may receive production issues from the support team.

For example:

> "Customer placed an order, payment was successful, but order status is still pending."

The developer checks the order ID, payment status, application logs, database records, and external payment-service response.

The issue is investigated based on evidence rather than changing production data immediately.

---

36. Common Production Issues

Some common issues include:

Login failure: Authentication service or database connectivity problem.

Slow product search: Database query or indexing issue.

Payment pending: External payment service timeout or delayed response.

Order not created: Validation, database, or service failure.

Incorrect inventory: Concurrency or inventory-update problem.

The exact root cause must be determined using logs, monitoring, and database information.

---

37. Monitoring

Production monitoring helps the team identify issues quickly.

The team monitors:

- Application availability
- API response time
- Error rate
- Database health
- CPU and memory
- Order failures
- Payment failures

If an important metric crosses a configured threshold, the operations team can investigate.

---

38. Incident Management

When a serious production problem occurs, the issue is treated as an incident.

The team identifies the impact, investigates the cause, provides a workaround or fix, and monitors the system after the change.

After resolving the incident, the team may perform a root-cause analysis to prevent the same issue from happening again.

---

39. Knowledge Required for a New Developer

A new developer joining this project should first understand the business workflow.

After that, the developer should study the architecture, source-code structure, APIs, database tables, and major modules.

The developer should then understand Git, testing, deployment, logging, monitoring, and production-support procedures.

The goal of KT is not just to explain the code but to make the new developer capable of independently working on project tasks.

---

40. Final KT Summary

The E-Commerce Order Management System manages the complete shopping and order process.

The basic workflow is:

```
Customer
   ↓
Login
   ↓
Browse Products
   ↓
Add to Cart
   ↓
Checkout
   ↓
Payment
   ↓
Order Creation
   ↓
Inventory Update
   ↓
Shipment
   ↓
Delivery
```

From a developer's perspective, understanding the project requires knowledge of both the **business flow and technical implementation**.

The most important areas for KT are the order lifecycle, payment processing, inventory management, REST APIs, database structure, source-code architecture, testing, deployment, monitoring, and production troubleshooting.

A successful KT should enable the new developer to take a requirement, understand which modules are affected, make the required code changes, test them, raise a pull request, support deployment, and investigate issues when the application is running in production.